

NOIP 模拟赛

Neko Olympiad in Introductory Problems

Mirekintoc Void

时间：2026 年 9 月 8 日 08:00 ~ 13:00

题目名称	肯德基	麦当劳	汉堡王	德克士
题目类型	传统型	传统型	传统型	传统型
目录	kfc	mdl	bk	dicos
可执行文件名	kfc	mdl	bk	dicos
输入文件名	kfc.in	mdl.in	bk.in	dicos.in
输出文件名	kfc.out	mdl.out	bk.out	dicos.out
每个测试点时限	1.0 秒	1.5 秒	1.5 秒	1.5 秒
内存限制	512 MiB	512 MiB	512 MiB	512 MiB
测试点数目	20	20	20	20
测试点是否等分	是	是	是	是

提交源程序文件名

对于 C++ 语言	kfc.cpp	mdl.cpp	bk.cpp	dicos.cpp
-----------	---------	---------	--------	-----------

编译选项

对于 C++ 语言	-O2 -std=c++14 -static
-----------	------------------------

注意事项（请仔细阅读）

1. 文件名（程序名和输入输出文件名）必须使用英文小写。
2. main 函数的返回值类型必须是 int，程序正常结束时的返回值必须是 0。
3. 因违反以上两点而出现的错误或问题，申诉时一律不予受理。
4. 若无特殊说明，结果的比较方式为全文比较（过滤行末空格及文末回车）。
5. 选手提交的程序源文件必须不大于 100KB。
6. 程序可使用的栈空间内存限制与题目的内存限制一致。
7. 只提供 Linux 格式附加样例文件。
8. 评测时采用的机器配置为：Windows 11, i7-12700KF @3.60GHz, 32G, Lemonlime。使用的编译器版本为：x86_64-pc-msys GCC 13.3.0。上述时限以此配置为准。
9. 本场难度较易，请 AK 的同学不要大声喧哗。
10. 题目按照出题人在对应商家点外卖次数从多到少排序。

肯德基 (kfc)

【题目描述】

肯德基总部决定把 N 家门店接入一张新的“炸鸡调度网”。为了防止一份鸡块沿着环路转到冷掉，总部规定这张网络必须恰好是一棵树：所有门店互相可达，并且没有环。

第 i 家门店有一个正整数忙碌系数 A_i 。如果它在网络中直接连接了 d_i 家门店，那么它要同时处理 d_i 份配送消息、 d_i 个群聊以及若干没有人想看的日报，总部把它的工作代价定义为

$$A_i d_i^2.$$

总部不关心具体哪两家店连在一起，只希望所有门店的总代价尽可能小。

形式化题意： 设 $\mathcal{T}_N$ 为顶点集为 $1, 2, \dots, N$ 的所有树的集合，求

$$\min_{T \in \mathcal{T}_N} \sum_{i=1}^N A_i \deg_T^2(i).$$

【输入格式】

从文件 `kfc.in` 中读入数据。

第一行一个整数 N 。

第二行 N 个整数 $A_1, A_2, \dots, A_N$ 。

【输出格式】

输出到文件 `kfc.out` 中。

输出一个整数，表示最小可能总代价。

【样例 1 输入】

```
1 4
2 3 2 5 2
```

【样例 1 输出】

```
1 24
```

【样例 1 解释】

取三条边 $(1, 2), (2, 4), (4, 3)$ ，各点度数依次为 $1, 2, 1, 2$ ，总代价为 $1^2 \times 3 + 2^2 \times 2 + 1^2 \times 5 + 2^2 \times 2 = 24$ 。

可以证明不存在更小的方案。

【样例 2 输入】

```
1 5
2 1 1 1 1 1
```

【样例 2 输出】

```
1 14
```

【样例 3】

见选手目录下的 *kfc/kfc3.in* 与 *kfc/kfc3.ans*。
该样例满足测试点 1 的数据范围。

【样例 4】

见选手目录下的 *kfc/kfc4.in* 与 *kfc/kfc4.ans*。
该样例满足测试点 3 的数据范围。

【样例 5】

见选手目录下的 *kfc/kfc5.in* 与 *kfc/kfc5.ans*。
该样例满足测试点 9 的数据范围。

【样例 6】

见选手目录下的 *kfc/kfc6.in* 与 *kfc/kfc6.ans*。
该样例满足测试点 11 的数据范围。

【样例 7】

见选手目录下的 *kfc/kfc7.in* 与 *kfc/kfc7.ans*。
该样例满足测试点 13 的数据范围。

【数据范围】

对于所有测试数据，保证： $2 \leq N \leq 2 \times 10^5$ ， $1 \leq A_i \leq 10^9$ ，答案小于 2^{63} 。

测试点编号	$N \leq$	特殊性质
1 ~ 2	10	无
3 ~ 8	5000	
9 ~ 10	2×10^5	A
11 ~ 12		B
13 ~ 20		无

特殊性质 A：保证 $3 \max_i A_i \leq 5 \min_i A_i$ 。

特殊性质 B：保证每个 A_i 均为 1 或 10^9 ，且至少存在一个 $A_i = 1$ 。

麦当劳 (mdl)

【题目描述】

麦当劳有两套完全不同的内部层级系统。

第一套叫“排班树” T_1 ，第二套叫“冰淇淋机报修树” T_2 。两棵树都有 N 名员工，编号为 $1, 2, \dots, N$ ，并且都以 1 号员工为根。至于为什么修冰淇淋机也需要一棵组织树，没有人问过。

现在要拉一个群。两名不同员工 u, v 能同时进群，当且仅当：

- 在 T_1 中，一人是另一人的祖先；
- 在 T_2 中，两人互相都不是对方的祖先。

这里，若 u 位于从根到 v 的路径上，则称 u 是 v 的祖先。树的根节点深度定义为 0，其余节点的深度为其父亲节点的深度加 1。

请你求这个群最多能拉多少人。

形式化题意： 记 $u \preceq_T v$ 表示 u 是 v 在树 T 中的祖先。求最大的 $|S|$ ，满足对任意不同的 $u, v \in S$,

$$\left(\left(u \preceq_{T_1} v \right) \vee \left(v \preceq_{T_1} u \right) \right) \wedge \neg \left(\left(u \preceq_{T_2} v \right) \vee \left(v \preceq_{T_2} u \right) \right).$$

【输入格式】

从文件 `mdl.in` 中读入数据。

第一行一个整数 t ，表示测试用例组数。

对于每组测试数据：

- 第一行一个整数 N ；
- 第二行 $N - 1$ 个整数 $a_2, a_3, \dots, a_N$ ，其中 a_i 表示第 i 个节点在 T_1 中的父亲编号；
- 第三行 $N - 1$ 个整数 $b_2, b_3, \dots, b_N$ ，其中 b_i 表示第 i 个节点在 T_2 中的父亲编号。

【输出格式】

输出到文件 `mdl.out` 中。

对每组测试数据输出一行一个整数，表示答案。

【样例 1 输入】

```
1 1
2 5
3 1 2 3 4
4 1 1 1 1
```

【样例 1 输出】

1 4

【样例 1 解释】

可以选择员工 2,3,4,5。他们在第一棵树中两两存在祖先关系，在第二棵树中两两均不存在祖先关系，因此答案至少为 4；可以证明不存在更大的合法集合。

【样例 2 输入】

1 1
2 7
3 1 1 3 4 4 5
4 1 2 1 4 2 5

【样例 2 输出】

1 3

【样例 2 解释】

一种最优方案是选择员工 3,4,6。

【样例 3】

见选手目录下的 *mdl/mdl3.in* 与 *mdl/mdl3.ans*。
该样例满足测试点 1 的数据范围。

【样例 4】

见选手目录下的 *mdl/mdl4.in* 与 *mdl/mdl4.ans*。
该样例满足测试点 3 的数据范围。

【样例 5】

见选手目录下的 *mdl/mdl5.in* 与 *mdl/mdl5.ans*。
该样例满足测试点 9 的数据范围。

【样例 6】

见选手目录下的 *mdl/mdl6.in* 与 *mdl/mdl6.ans*。
该样例满足测试点 10 的数据范围。

【样例 7】

见选手目录下的 `mdl/mdl7.in` 与 `mdl/mdl7.ans`。
该样例满足测试点 11 的数据范围。

【样例 8】

见选手目录下的 `mdl/mdl8.in` 与 `mdl/mdl8.ans`。
该样例满足测试点 13 的数据范围。

【样例 9】

见选手目录下的 `mdl/mdl9.in` 与 `mdl/mdl9.ans`。
该样例满足测试点 15 的数据范围。

【数据范围】

对于所有测试数据，保证： $1 \leq t \leq 3 \times 10^5$ ， $2 \leq N \leq 3 \times 10^5$ ， $1 \leq a_i < i$ ， $1 \leq b_i < i$ 。
令 d_1, d_2 分别表示 T_1, T_2 的最大深度。保证单个输入文件中 $\sum N \leq 3 \times 10^5$ 。

测试点编号	$\sum N \leq$	$d_1 \leq$	$d_2 \leq$	特殊性质
1 ~ 2	20	$N - 1$	$N - 1$	无
3 ~ 8	3000	$N - 1$	$N - 1$	
9	3×10^5	$N - 1$	$N - 1$	A
10		20	$N - 1$	无
11 ~ 12		$N - 1$	$N - 1$	B
13 ~ 14		$N - 1$	2	无
15 ~ 20		$N - 1$	$N - 1$	无

特殊性质 A：保证在 T_1 中任意两个节点均有一个是另一个的祖先。
特殊性质 B：保证在 T_2 中，除根以外的每个节点至多有一个儿子。

汉堡王 (bk)

【题目描述】

汉堡王有一条长度为 N 的出餐传送带，位置从 1 到 N 编号。每个位置恰好放着一个餐盒；有些餐盒里有汉堡，有些是空的。保证至少有一个汉堡，也至少有一个空盒。

员工每次只能进行如下操作：选择一个装有汉堡的餐盒，以及与它相邻的一个空餐盒，把汉堡移到这个空餐盒中。换句话说，一次操作会把相邻的 **01** 变成 **10**，或者把 **10** 变成 **01**。

今天国王下令：必须恰好搬 K 次。求经过恰好 K 次操作后，可能出现多少种不同的汉堡摆放状态。若两个状态至少有一个位置是否放有汉堡不同，则认为它们不同。

形式化题意： 令 R_t 为从初始状态出发恰好进行 t 次合法相邻移动后可得到的所有二进制串的集合，求

$$|R_K| \bmod (10^9 + 7).$$

【输入格式】

从文件 `bk.in` 中读入数据。

第一行两个整数 N, K 。

第二行 N 个整数 $A_1, A_2, \dots, A_N$ 。 $A_i = 1$ 表示第 i 个位置有汉堡， $A_i = 0$ 表示为空。

【输出格式】

输出到文件 `bk.out` 中。

输出一个整数，表示答案对 $10^9 + 7$ 取模后的结果。

【样例 1 输入】

```
1 4 1
2 1 0 1 0
```

【样例 1 输出】

```
1 3
```

【样例 1 解释】

恰好移动一次后，可能得到 **0110**、**1001**、**1100**，共 3 种状态。

【样例 2 输入】

```
1 4 2
2 1 0 1 0
```

【样例 2 输出】

```
1 2
```

【样例 2 解释】

恰好移动两次后，只可能得到 **0101** 和 **1010** 两种状态。特别地，回到初始状态也是允许的：可以先进行一次合法移动，再将同一个汉堡移回原位。

【样例 3】

见选手目录下的 *bk/bk3.in* 与 *bk/bk3.ans*。
该样例满足测试点 1 的数据范围。

【样例 4】

见选手目录下的 *bk/bk4.in* 与 *bk/bk4.ans*。
该样例满足测试点 3 的数据范围。

【样例 5】

见选手目录下的 *bk/bk5.in* 与 *bk/bk5.ans*。
该样例满足测试点 8 的数据范围。

【样例 6】

见选手目录下的 *bk/bk6.in* 与 *bk/bk6.ans*。
该样例满足测试点 11 的数据范围。

【样例 7】

见选手目录下的 *bk/bk7.in* 与 *bk/bk7.ans*。
该样例满足测试点 13 的数据范围。

【样例 8】

见选手目录下的 *bk/bk8.in* 与 *bk/bk8.ans*。
该样例满足测试点 15 的数据范围。

【数据范围】

对于所有测试数据，保证： $2 \leq N \leq 1500$ ， $1 \leq K \leq 1500$ ， $A_i = 0$ 或 $A_i = 1$ ，初始状态中至少有一个 0 和一个 1。

测试点编号	$N \leq$	$K \leq$	特殊性质
1 ~ 2	12	1500	无
3 ~ 7	100	1500	
8 ~ 10	1500	30	
11 ~ 12	1500	1500	A
13 ~ 14			B
15 ~ 20			无

特殊性质 A：保证 $\min(\sum A_i, N - \sum A_i) \leq 20$ 。
特殊性质 B：保证至多有一个位置 i 满足 $A_i \neq A_{i+1}$ 。

德克士 (dicos)

【题目描述】

德克士有 N 家门店和 M 条双向配送道路。每条道路的通行时间只有两种可能： a 或 b ，其中 $a < b$ 。道路网络连通，没有自环，也没有重边。

财务部门决定“优化”道路：删掉若干道路，但要求剩余道路仍能让任意两家门店互相到达，并且剩余道路的通行时间总和尽可能小。于是，剩下的道路必须构成原图的一棵最小生成树。

老板的办公室在 1 号门店。对于每个 $1 \leq i \leq N$ ，老板都希望在所有满足上面条件的删路方案中，使从 1 到 i 的剩余道路总时间尽可能小。

你需要分别求出每个 i 对应的最小值。不同的 i 可以采用不同的最小生成树，这一点很重要，也很符合临时改需求的传统。

形式化题意： 设 $\mathcal{M}(G)$ 为图 G 的所有最小生成树的集合。对每个 $i = 1, 2, \dots, N$ ，求

$$\text{ans}_i = \min_{T \in \mathcal{M}(G)} \text{dist}_{T(1,i)}.$$

【输入格式】

从文件 `dicos.in` 中读入数据。

第一行四个整数 N, M, a, b 。

接下来 M 行，第 i 行三个整数 u_i, v_i, c_i ，表示 u_i, v_i 之间有一条通行时间为 c_i 的双向道路。

【输出格式】

输出到文件 `dicos.out` 中。

输出一行 N 个整数，其中第 i 个整数为 ans_i 。

【样例 1 输入】

```
1 5 5 20 25
2 1 2 25
3 2 3 25
4 3 4 20
5 4 5 20
6 5 1 20
```

【样例 1 输出】

```
1 0 25 60 40 20
```

【样例 1 解释】

所有最小生成树的边权和均为 85。虽然原图中 $1 \rightarrow 2 \rightarrow 3$ 的长度只有 50，但保留这两条权为 25 的道路会使剩余道路无法同时成为最小生成树，因此 3 号门店的答案为 60。

【样例 2 输入】

```
1 6 7 13 22
2 1 2 13
3 2 3 13
4 1 4 22
5 3 4 13
6 4 5 13
7 5 6 13
8 6 1 13
```

【样例 2 输出】

```
1 0 13 26 39 26 13
```

【样例 3】

见选手目录下的 *dicos/dicos3.in* 与 *dicos/dicos3.ans*。
该样例满足测试点 1 的数据范围。

【样例 4】

见选手目录下的 *dicos/dicos4.in* 与 *dicos/dicos4.ans*。
该样例满足测试点 3 的数据范围。

【样例 5】

见选手目录下的 *dicos/dicos5.in* 与 *dicos/dicos5.ans*。
该样例满足测试点 9 的数据范围。

【样例 6】

见选手目录下的 *dicos/dicos6.in* 与 *dicos/dicos6.ans*。
该样例满足测试点 11 的数据范围。

【样例 7】

见选手目录下的 *dicos/dicos7.in* 与 *dicos/dicos7.ans*。
该样例满足测试点 13 的数据范围。

【数据范围】

对于所有测试数据，保证： $2 \leq N \leq 70$ ， $N - 1 \leq M \leq 200$ ， $1 \leq a < b \leq 10^7$ ， $1 \leq u_i, v_i \leq N$ ， $u_i \neq v_i$ ， $c_i = a$ 或 $c_i = b$ ，图连通且无自环、无重边。

测试点编号	$N \leq$	$M \leq$	特殊性质
1 ~ 2	12	30	无
3 ~ 8	18	60	
9 ~ 10	70	200	A
11 ~ 12			B
13 ~ 20			无

特殊性质 A：保证任意两个仅经过权值为 a 的边即可互相到达的节点之间，都存在一条边数不超过 2 且所有边权均为 a 的路径。
特殊性质 B：保证 $2b \geq 3a$ 。